

AppleSupport AI Agent Report

1. Problem framing

AppleSupport handles short, noisy, multi-turn customer-support messages. A useful agent for this brand should do three things reliably: identify the broad issue, suggest a next step consistent with AppleSupport's historical behavior, and avoid pretending that a security, fraud, legal, or safety case is routine.

For this prototype, "good" means:

- intent labels are understandable and stable enough for macro-F1 evaluation;
- replies are actionable, concise, privacy-aware, and grounded in historical AppleSupport resolutions;
- high-risk cases are routed to a person with an explicit reason;
- every prediction can be inspected and reproduced locally.

The scope is intentionally narrow: one brand, a 2,000-example development slice, nine intent labels, PostgreSQL interaction logging, and a Flask interface. I chose not to build a production authentication system, automatic outbound messaging, payments, a fine-tuned language model, or a fully autonomous agent. The prototype does not claim to contact Apple or resolve an account itself.

2. Data and pipeline

The source is the Customer Support on Twitter dataset supplied for the assignment. AppleSupport replies are identified by `author_id == AppleSupport`; customer messages are linked through the dataset's reply relationships. The current generated slice contains 2,000 linked inbound examples and a 200-row golden-set template. The golden set must be manually labelled before quality metrics are meaningful.

The pipeline is:

```
customer message -> intent baseline -> escalation baseline -> draft reply -> PostgreSQL log
```

The Flask app exposes `POST /api/analyze`, `GET /api/history`, `GET /api/escalations`, and `GET /api/metrics`. A separate read-only `/database` page makes the stored records easy to inspect without exposing SQL credentials.

3. Results and baselines

Current evidence

Check	Current result
AppleSupport rows found in source	106,860
Development examples generated	2,000
Development examples with linked historical replies	2,000
Golden evaluation examples prepared	200
Manually labelled golden examples	0 / 200
Retrieved historical replies per analysis	3

The last row is important: classification accuracy, macro-F1, escalation precision/recall, and reply-quality scores are **not reported yet** because the golden labels are still blank. Reporting a score before labelling would be misleading.

Required comparison plan

Once `data/golden_eval.csv` is labelled, run the same held-out examples through:

1. **Trivial baseline:** always predict the most frequent intent in the training portion; always choose `auto-handle` for escalation.

2. **Simple baseline:** TF-IDF word and character n-grams with logistic regression for intent; the current transparent risk-term rules for escalation.
3. **Prototype:** the current keyword intent rules, risk-term escalation rules, TF-IDF historical-reply retrieval, and grounded response layer.

Report macro-F1 rather than only accuracy because intent frequencies will be uneven. For escalation, report precision, recall, and a confusion matrix. For replies, use a blinded human rubric covering correctness, grounding, actionability, privacy, and tone. Compare an LLM judge with human labels on at least a sample and report agreement, not just the judge's score.

4. Failure analysis

These are the five highest-risk failure modes to test and record during golden-set review. The examples below are representative messages from the project data or its test question bank.

1. **Keyword collision.** A message such as "my battery is draining" can match both software troubleshooting and repair/warranty because battery is used in both vocabularies. Hypothesis: mutually exclusive keyword ordering is too brittle. Fix: labelled examples plus a TF-IDF or embedding classifier with confidence thresholds.
2. **Link-only or context-dependent tweets.** Real rows include messages such as @AppleSupport <https://t.co/NV0yucs01B>, where the meaning is in an image or an earlier turn. Hypothesis: single-message classification loses thread context. Fix: reconstruct the conversation window and include parent messages or image metadata.
3. **Generic routing replies are not resolutions.** Historical replies such as "Let's take a closer look into this issue... join us in a DM" are safe routing behavior but do not prove that the underlying problem was solved. Hypothesis: reply grounding can learn to overuse DM language. Fix: separate routing replies from substantive resolution examples.
4. **Security language is implicit.** "I do not recognize this Apple charge" may indicate fraud without containing the current fraud or chargeback terms. Hypothesis: exact risk terms produce false negatives. Fix: add annotated security examples and a conservative semantic risk classifier.
5. **Intent overlap and ambiguous requests.** "This Apple Account is not valid or not supported" is account-related, but the root cause could be region, activation, billing, or service eligibility. Hypothesis: one-label taxonomy hides uncertainty. Fix: allow other or multi-label tags and escalate low-confidence cases.

5. What is misleading about my headline number?

The most tempting headline is a future number such as "87% reply confidence" or a high intent accuracy. That number would be misleading if it came from keyword coverage, an LLM judge alone, or an unbalanced sample. It would not establish that the agent solves customer problems, handles rare security cases, or generalizes to unseen threads. The current 87% value is explicitly labelled **baseline confidence** in the UI; it is a presentation baseline, not a measured accuracy claim.

A trustworthy headline should state the sample size, label policy, class distribution, confidence interval where practical, and whether the metric is human-verified. The report should also show the denominator and at least five failures rather than presenting one aggregate number.

6. What I would do with one more week

- Label and adjudicate all 200 golden examples, including a second review for ambiguous and high-risk cases.
- Add a chronological train/test split to reduce leakage from near-duplicate conversations.
- Compare TF-IDF retrieval with a sentence-embedding retrieval model and measure evidence quality.
- Add a confidence threshold that escalates uncertain predictions instead of forcing an intent.
- Build the reply rubric and measure judge-human agreement.
- Add PII redaction, structured audit logs, rate limiting, and tests for API/database failures.

- Add authentication and role-based access to the database viewer before exposing it to a wider operations team.

7. Decision log

1. Chose AppleSupport because it has 106,860 rows and repeated support workflows.
2. Restricted the first slice to one brand to make evaluation and failure analysis defensible.
3. Used the dataset's reply relationships instead of assuming adjacent CSV rows form a conversation.
4. Kept nine broad intents rather than inventing many fine-grained labels before annotation.
5. Included other/general inquiry behavior for messages that do not match a stable intent.
6. Started with transparent keyword rules so every prediction can be explained live.
7. Used explicit high-risk terms for the first escalation policy because false reassurance is worse than review.
8. Prepared 200 golden rows because the assignment asks for 150-250 hand-labelled examples.
9. Kept golden labels blank until a human annotates them, avoiding fabricated evaluation claims.
10. Used historical AppleSupport replies as evidence rather than presenting generic generated text as brand truth.
11. Added database logging so predictions, reasons, and timestamps are auditable; production uses Render PostgreSQL and local development uses SQLite.
12. Added a read-only database viewer so reviewers can inspect records without direct SQL access.
13. Kept the customer UI separate from the agent dashboard so end-user support and operator review have different workflows.
14. Excluded the assignment PDF, virtual environment, caches, and local database from Git.
15. Added GitHub Actions and Render configuration for reproducible deployment, while keeping database credentials out of the repository.